STUDIO IA — Mission 004 : restauration neuronale locale

Version du cahier : 1.0 — 7 novembre 2026. Statut : **préparé pour validation ; développement lourd non lancé**. Socle : STUDIO IA CPU 0.3.0a2, issu des huit commits M003 préservés. Cibles : Windows x64, puis Linux x64. Android reste une interface de pilotage GitHub, pas une plateforme d'exécution qualifiée.

1. Objectif et conditions d'entrée

Ajouter des moteurs locaux réellement exécutés derrière l'interface existante, sans perdre les garanties de préservation, d'annulation et de vérification. Le parcours reste : importer → analyser → proposer → prévisualiser → confirmer → traiter → vérifier → comparer → exporter.

Avant tout chantier lourd M004 : sauvegarde et restauration du bundle M003 prouvées ; code publié sur branche dédiée ; E2E CPU et CI Windows/Linux verts ; au moins une distribution Windows x64 lancée sur runner natif ; anomalies prioritaires de la consolidation closes. Les portes absentes restent BLOCKED, même si le cahier est complet. Aucune fusion dans main sans autorisation.

Le présent cahier n'autorise ni l'installation d'un modèle/driver sur un appareil personnel, ni une dépense, ni un runner GPU payant, ni l'envoi de médias privés. Il ne déclenche aucune installation de moteur neuronal dans la consolidation.

2. Périmètre produit

Trois capacités sont prévues, activables séparément :

1. Super-résolution/restauration d'images vidéo avec Real-ESRGAN, en conservant les timestamps et toutes les pistes audio prises en charge.

2. Restauration/agrandissement d'animation avec une variante Anime4K précisément identifiée. Le nom seul ne garantit pas une technique neuronale.

3. Interpolation temporelle RIFE, uniquement après choix explicite de l'utilisateur. Conserver la durée normale ; aucun ralenti implicite, aucun passage invisible à 60 fps.

Exclus : text-to-video, scènes nouvelles, montage multi-pistes, remplacement de personnes/décors, entraînement, fine-tuning, face enhancement automatique, synchronisation cloud, API commerciale et APK prétendument fonctionnelle. Un agrandissement 4K décrit une résolution, pas une récupération fiable de détails.

3. État des sources vérifiées et décisions de moteurs

Les sources officielles ont été lues pour préparer ce cahier. Aucun des moteurs, binaires ou poids ci-dessous n'a été installé ou exécuté pendant la consolidation. Les performances de leurs README ne sont pas des mesures STUDIO IA.

Famille	Chemin candidat	Conditions à établir avant intégration
Real-ESRGAN	Exécutable NCNN/Vulkan officiel, ou backend Video2X qualifié	Version exacte, aide locale, modèle pris en charge, API/GPU, licence et hashes
Real-ESRGAN alternatif	Processus Python/PyTorch isolé	Seulement si besoin démontré ; dépendances/poids sûrs, coût mémoire et runtime documentés
Anime4K	GLSL via libplacebo/Video2X ou autre backend réellement disponible	Shader exact, version, chaîne de shaders, variante CNN ou classique, compatibilité GPU
RIFE	NCNN/Vulkan officiel, ou backend Video2X qualifié	Modèle exact, convention de frames terminales, cadence/PTS, mémoire et arrêt

Real-ESRGAN publie une voie NCNN portable ne nécessitant pas CUDA/PyTorch, avec des différences de fonctions et de tuilage par rapport à l'implémentation Python. Ne pas transposer les options d'un backend à l'autre.

Anime4K décrit des upscalers CNN, des shaders de défloutage et des débruiteurs classiques/CNN. L'interface doit distinguer neural_cnn de classic_shader selon la chaîne réellement exécutée. Ne pas déclarer toute variante Anime4K « IA ».

Video2X a une réécriture C/C++ depuis la version 6. Son README décrit les shaders GLSL, NCNN/Vulkan, Real-ESRGAN, Real-CUGAN et RIFE. L'API releases consultée renvoie la version **6.4.0**, publiée le 24 janvier 2025. Ses notes indiquent un sous-ensemble de modèles RIFE embarqués. Ce constat documentaire n'établit aucune compatibilité locale ; la version et --help du binaire installé feront autorité.

Video2X reste une option d'orchestration, pas une dépendance obligatoire du socle. Choisir une voie principale seulement après le prototype court et l'audit des licences ; éviter d'installer deux piles complètes sans besoin mesuré.

4. Licences, provenance et sécurité des modèles

Constat sur les licences de code consultées : Real-ESRGAN BSD-3-Clause ; Anime4K MIT ; ECCV2022-RIFE MIT ; Video2X AGPL-3.0. Les licences des poids, shaders, convertisseurs, runtimes et binaires restent à vérifier **pour chaque artefact**. La licence du dépôt ne suffit pas à supposer tous les droits sur tous les poids. Ne pas affirmer qu'un appel en sous-processus annule des obligations AGPL/LGPL.

Chaque entrée du registre doit inclure : identifiant, famille/opération, nom et version, backend compatible, URL officielle, release/commit, licence du code et des poids, chemin local sélectionné, taille, SHA-256 attendu et calculé, fichiers associés (.param/.bin par exemple), exigences GPU/API documentées et mesures locales. Le manifeste ne contient aucun secret.

États distincts : absent → détecté → intégrité vérifiée → compatible → test court réussi → vidéo complète validée. Un hash correct ne prouve ni l'innocuité ni la qualité d'un modèle. Un moteur présent ne devient pas « opérationnel » par détection.

Pas de téléchargement à l'ouverture ou pendant un traitement. Un téléchargement futur doit être annoncé avec provenance, licence, taille et destination, puis explicitement déclenché. Import local toujours disponible. Si l'éditeur ne fournit pas d'empreinte/signature, consigner cette absence ; ne pas inventer une preuve d'authenticité. Privilégier une construction reproductible depuis un commit fixé ou une source d'intégrité indépendante avant qualification.

Les formats PyTorch pickle/PTH peuvent contenir du code : ne charger que des poids officiels vérifiés via une méthode de chargement sûre effectivement supportée. Ne pas promettre safetensors ou une conversion sûre sans implémentation testée. Les moteurs s'exécutent dans des processus possédés par l'application, sans shell, avec arguments validés, délais, budget mémoire et arrêt du groupe. Un sous-processus n'est pas présenté comme une sandbox de sécurité complète.

5. Qualification matérielle et stratégie de ressources

Sonder : OS/architecture, CPU et instructions requises par le binaire, GPU exact, driver, API Vulkan réellement utilisable, mémoire disponible et version du runtime. Un GPU listé dans le système n'est pas une preuve de calcul Vulkan fonctionnel.

Pour chaque couple moteur/modèle : exécuter ultérieurement un minuscule traitement de qualification ; mesurer durée, résolution, pic mémoire si mesurable, code de sortie et décodage. Les limites mémoire/VRAM doivent venir de la documentation vérifiée ou de mesures, jamais d'une valeur universelle inventée.

Une seule tâche intensive par défaut. Paramètres bornés de tuilage, threads, échelle et budget disque. Estimer et afficher le stockage temporaire à partir du nombre de frames, dimensions et format intermédiaire ; prévoir une marge mesurée. Préférer des lots bornés à une extraction illimitée de toute la vidéo.

En cas d'OOM : annuler proprement, ne pas finaliser, proposer un lot/une tuile plus petite si le backend le permet. Pas de relance infinie. Si aucun backend neuronal n'est compatible : proposer explicitement FFmpeg CPU classique ; résultat IA SKIP.

6. Architecture et contrats à ajouter ultérieurement

Conserver les modules CPU, jobs SQLite, UI et CLI communs. Étendre le contrat EngineAdapter préparé en M003 : probe → validate → estimate → preview → run → cancel. L'adaptateur retourne capacités, identité du moteur et modèle, progression mesurée ou indéterminée, journal local, étapes accomplies et fichiers possédés.

TreatmentRequest/Plan version 2 : requête utilisateur, source canonique, empreinte source, source MediaInfo, graphe ordonné d'étapes, moteur/modèle figés, échelle, politique de cadence, budget et politique audio/couleurs. Une migration explicite doit continuer à lire les jobs CPU version 1 ; ne pas réinterpréter un ancien plan.

Représenter les timestamps par entiers et time base rationnelle ; conserver une table frame_id → PTS → durée → fichier/lot/hash quand le backend travaille en images. Définir si le backend accepte une vidéo ou des frames ; ne pas prétendre reprendre au dernier frame lorsque son API ne permet pas de checkpoint vérifié.

Persister le plan figé et l'identité de chaque moteur. Les états existants restent compatibles ; détailler l'étape (extraction, inférence, assemblage, vérification) sans marquer SUCCEEDED avant toutes les assertions et la publication effective. Un journal VERIFIED_NOT_COMMITTED n'est jamais présenté comme un succès d'export.

Les graphiques de traitement sont acycliques et limités aux opérations autorisées. Ne jamais permettre à un manifeste de fournir une commande shell ou un script. Les chemins de modèles et médias restent locaux et sont validés.

7. Chaîne vidéo et ordre de traitement

Chaîne initiale proposée, à confirmer sur le corpus : analyse → éventuel débruitage léger explicitement choisi → restauration/super-résolution → interpolation opt-in → réassemblage/encodage → audio/métadonnées → validation → publication sûre.

La super-résolution neuronale **remplace** le Lanczos de grossissement ; elle ne doit pas s'y ajouter aveuglément. Si un modèle natif ×4 produit une cible ×2 par réduction finale, l'indiquer dans le plan et dans la comparaison. Ne pas appliquer systématiquement Real-ESRGAN et Anime4K sur une même vidéo. Prévoir une comparaison séparée avant de proposer une combinaison.

Les formats HDR/BT.2020, >8 bits, rotation, entrelacement et SAR non carré restent refusés tant qu'une politique spécifique n'a pas été développée et testée. Aucun tone mapping, désentrelacement ou changement de gamme invisible. Sous-titres, données et multi-vidéo restent sous la politique de refus explicite du MVP.

Conserver toutes les pistes audio autorisées. L'audio doit être mappé par STUDIO IA, même si l'outil neuronal l'abandonne. Ne pas utiliser une option de ralenti qui supprime l'audio : le README RIFE signale précisément ce comportement pour certains usages.

8. Cadence, VFR et interpolation RIFE

Pour une super-résolution sans interpolation : même ensemble de PTS, même nombre de frames, même durée vidéo dans la tolérance d'encodage documentée.

Pour RIFE : mode initial ×2 CFR explicitement demandé. Exemple contractuel : 200 frames à 25 fps couvrant 8 s → 400 frames à 50 fps couvrant 8 s, sans accélération. Si le backend retourne 2N−1 frames, compléter l'intervalle final par maintien du dernier frame, avec compteur/raison enregistrés ; ne pas inventer une image future.

Chaque frame original reste ancré à son timestamp. Détecter les coupes de scène avec une méthode documentée et contrôlable ; ne pas interpoler un morphing entre deux scènes sans rapport. Préférer maintien/répétition explicite à une transition inventée. Tester aussi occlusions, mouvements rapides et plans statiques.

VFR : conserver la table de timestamps pour la super-résolution. Pour l'interpolation, soit un backend respecte réellement ces timestamps, soit proposer une conversion CFR annoncée et confirmée avec règle de rééchantillonnage précise ; sinon refuser. Ne jamais choisir un fps à partir d'un arrondi arbitraire de avg_frame_rate.

9. Interface et fonctionnement hors ligne

Ajouter un choix de moteur compréhensible : CPU classique / Real-ESRGAN / Anime4K qualifié / RIFE interpolation. Les choix non compatibles sont désactivés avec une raison concrète et une action de diagnostic, jamais des boutons factices.

Afficher le modèle, la technique exacte, la taille/cadence résultante, les étapes, le stockage estimé et les risques de détails inventés/scintillement. Proposer un échantillon court issu du fichier courant et une comparaison à la source et au profil CPU. Le lancement du fichier complet doit rester explicite.

La prévisualisation emploie les paramètres et modèles du plan complet ; elle ne constitue pas une preuve pour toute la vidéo. Comparaison au même instant/PPS disponible, curseur et accès aux frames voisins. Prévoir la lecture temporelle A/B si elle peut être testée sans bloquer le parcours minimal.

Progression fondée sur frames réellement traitées ou progression native vérifiée. Sinon barre indéterminée. Ne pas annoncer de temps restant avant des mesures suffisantes. Annuler à chaque étape, conserver le journal et nettoyer uniquement les fichiers possédés. Relancer depuis l'original ou un checkpoint vérifié explicite.

Après préinstallation, aucune connexion Internet ne doit être nécessaire. Test d'exécution avec réseau désactivé obligatoire, y compris modèles/aperçus. Aucun compte, clé API, crédit ou téléversement implicite.

10. Corpus et mesures de qualité

Conserver F01–F08 et toutes les régressions CPU/GUI. Ajouter :

ID	Source contrôlée	Assertions et examen
N01	F01 court avec audio	vraie inférence, décodage, taille, durée, audio, source intacte
N02	Animation synthétique à lignes/texte/contours francs	halos, lisibilité, disparition de traits, stabilité temporelle
N03	Référence haute résolution dégradée de façon reproductible	comparaison source dégradée, CPU et NN alignés ; métriques interprétées
N04	Mouvement rapide et occlusions, référence haute cadence	RIFE ×2, ancrages, nombre de frames, artefacts de mouvement
N05	Coupe de scène, fixe et panoramique	pas de morphing entre scènes ; arrêt de mouvement respecté
N06	VFR + audio retardé + plusieurs pistes audio	PTS/cadence selon politique, offset et durée audio
N07	Tuiles/limites mémoire/vidéo longue bornée	coutures, OOM, plafond mémoire, stockage et annulation
N08	Modèle absent/corrompu, moteur incompatible, crash	refus actionnable, aucun faux succès ni export partiel

Créer seulement des médias synthétiques ou du contenu à licence explicitement autorisée. Pas de visages privés ou vidéo utilisateur dans Git/CI. Un corpus de visages réels libres serait une extension soumise à ses droits, pas une promesse de reconnaissance ou d'amélioration identitaire.

PSNR/SSIM avec référence réellement alignée ; VMAF seulement si disponible et applicable. Ce sont des mesures de ressemblance, pas une vérité perceptuelle. Ne pas imposer un gain numérique universel à toute restauration sans référence. Documenter aussi les cas où le modèle est moins bon que le CPU ou l'original.

Inspection temporelle : petits caractères, traits fins, aplats, halos, saturation, scintillement, détails inventés, occlusions et doubles contours. Consigner les images/instants examinés et le mode de revue ; ne pas inventer un avis humain.

11. Matrice d'acceptation M004

ID	Critère de passage	Rejet si…
NN-01	Socle CPU et sauvegarde restaurable toujours verts	régression d'import/export/annulation
NN-02	Identité version/commandes réellement sondée	flags recopiés sans vérification locale
NN-03	Modèle/licence/provenance/SHA déclarés et vérifiés	poids inconnu ou hash inventé
NN-04	GPU/API ou CPU neuronal réellement compatible	détection seule présentée comme un test
NN-05	Real-ESRGAN traite N01 de bout en bout	mock, frame unique seulement ou audio perdu
NN-06	Anime4K N02, chaîne et nature CNN/classique tracées	étiquette IA non justifiée
NN-07	RIFE N04/N05 avec cadence, durée et ancrages corrects	accélération, coupe morphée ou frame count inexpliqué
NN-08	Audio complet, offset contrôlé, source hash inchangé	piste perdue ou source écrasée
NN-09	Aperçu et comparaison issus du fichier courant	démonstration préenregistrée
NN-10	Annulation/crash/OOM/disque traités sans faux succès	export partiel promu
NN-11	Exécution sans réseau après installation	dépendance à une API distante
NN-12	Preuves de qualité visuelle/temporalité, limites explicites	score marketing sans référence appropriée
NN-13	Build Windows natif et Linux qualifiés, licences incluses	compilation croisée supposée valide
NN-14	Branche/PR brouillon/CI et artefacts référencés par SHA	aucun commit distant vérifiable
NN-15	Bilans par moteur PASS/FAIL/SKIP, versions et hardware	SKIP transformé en « prêt »

Tolérances techniques initiales à justifier par le corpus : résolution exacte ; cadence rationnelle exacte pour le mode CFR choisi ; timestamps monotones ; durée vidéo à deux frames maximum ; dérive A/V introduite ≤ max(2 frames, 50 ms) quand les timestamps permettent cette mesure. Données insuffisantes : annoncer le contrôle non vérifiable ou refuser le mode, pas produire un PASS universel.

12. Lots de travail futurs et preuves attendues

Lot	Travail autorisé lors du lancement futur	Preuve avant passage
M4-0	Revalider consolidation, branche et licences	CI CPU/Windows, sauvegarde vérifiée
M4-1	Choisir un backend et un seul modèle de SR	doctor/version/aide, manifest/hashes/GPU
M4-2	Real-ESRGAN court, aperçu puis vidéo complète	N01/N03, décodage/audio/PTS/source
M4-3	Anime4K exact et profil animation	N02, chaîne de shaders, qualification CNN/classique
M4-4	RIFE ×2 explicite	N04/N05/N06, timing/audio et comparaison temporelle
M4-5	OOM/annulation/stockage, tests longs bornés	N07/N08, logs, absence de sortie partielle
M4-6	Packaging et revue qualité/licences	ZIP + SHA + lancement + bilan par moteur

Un commit et un rapport par lot. Pas de multi-modèles à grande échelle avant qu'un moteur ait traité une vraie vidéo. Les jobs GPU restent conditionnels au matériel effectivement fourni ; aucune dépense ni auto-provisionnement payant. Le socle CPU garde sa CI autonome. Ne pas créer de release dite stable ni fusionner.

Rapport obligatoire : date, branche/base/head local/distant, moteur/modèle/licences, hashes, matériel/runtime/API, commandes structurées, source/output synthétiques, temps/mémoire mesurés ou « non mesuré », assertions, annulation, run/artefacts, captures, PASS/FAIL/SKIP et limites. Les liens de runs n'existent qu'après exécution.

13. Sources officielles consultées

- Real-ESRGAN : https://github.com/xinntao/Real-ESRGAN et fichier LICENSE BSD-3-Clause.

- Anime4K : https://github.com/bloc97/Anime4K et fichier LICENSE MIT.

- RIFE : https://github.com/hzwer/ECCV2022-RIFE et fichier LICENSE MIT.

- RIFE NCNN : https://github.com/nihui/rife-ncnn-vulkan .

- Video2X : https://github.com/k4yt3x/video2x et release https://github.com/k4yt3x/video2x/releases/tag/6.4.0 .

- FFmpeg : https://ffmpeg.org/ffmpeg-filters.html et https://ffmpeg.org/ffprobe.html .

Les dates et URL de lecture sont fournies avec le cahier. Les branches upstream peuvent évoluer ; lors du lancement M004, fixer releases/commits et fichiers exacts avant toute installation. Aucun seuil de performance ni compatibilité GPU n'est annoncé comme acquis par la seule lecture de ces sources.